

主程总结版

游戏核心机制与玩法

- **类型：**结合吸血鬼like和卡牌的Rougelike游戏
 - 吸血鬼like：意味着玩家将在一个相对开放的地图上与大量敌人战斗，通过击败敌人获得经验、升级并选择能力，生存尽可能长的时间。
 - 卡牌：玩家通过使用卡牌（符咒）来释放技能，卡牌有循环使用的机制，增加了策略性
 - Roguelike: 意味着游戏具有随机生成性（关卡、敌人、卡牌、宝物等）、永久死亡（一旦失败就要重新开始）和资源管理等特性。
 - **主循环：**
 - 1. 选择武器
 - 2. 进入区域与关卡
 - 3. 打开地图并选择节点
 - 4. 进入选择的关卡
 - 5. 重复3和4
 - 6. 完成区域最后一关
 - 7. 进入下一区域，重复2-6，直到完成第三区域最后一关
 - 8. 结算奖励得分，游戏结束（或失败）
-

游戏元素拆分

1. 武器系统

- 策划案内容：
 - 无角色系统，每局开始前选择武器（Demo版本：桃木剑、三清铃）
 - 武器属性
 - Weapon_name -武器的名称
 - Weapon_order -武器的序号
 - Weapon_attack -武器攻击力
 - Weapon_type -武器类型
 - Weapon_damage -武器伤害类型
 - Weapon_speed -武器攻击速度
 - Weapon_range -武器攻击距离

- 存储实现方案：
 - 1. Unity自带的ScriptableObject类

```
// Assets/Scripts/WeaponData.cs
using UnityEngine;

[CreateAssetMenu(fileName = "NewWeapon", menuName = "Game/Weapon")]
public class WeaponData : ScriptableObject
{
    public string Weapon_name;
    public int Weapon_order;
    public float Weapon_attack;
    public string Weapon_type;
    public string Weapon_damage;
    public float Weapon_speed;
    public float Weapon_range;
}
```

- 在Project窗口中，右键 -> Create -> Game -> Weapon，创建桃木剑和三清铃的ScriptableObject实例，并填写属性。
 - 方案优点：创建流程简单，方便管理
 - 方案缺点：策划需要同步下载Unity管理项目，容易出现Git异常
- 2. Json存储
 - 方案优点：操作简单，excel表格配表轻松
 - 方案缺点：数据明文存储，容易被篡改，需要额外加密
- 武器选择：
 - 用一个简单的UI界面让玩家选择武器
 - 在游戏开始时，根据玩家选择的武器，遍历武器表得到对应数据加载到战斗管理器中
 - 通过战斗管理器管理一场战斗的相关元素
- 武器使用：
 - 武器的类型和伤害类型都使用枚举值进行区分
 - 近战、远程/物理伤害、法术伤害等
 - 近战武器：在攻击范围内检测敌人并造成伤害
 - 远程武器：生成子弹或投射物
 - 要运用对象池及时回收使用完的实例

2. 敌人系统

- 策划案内容：
 - 攻击方式：接近玩家或发送弹幕

- 部分敌人会有特殊的技能 `Enemy_skill`
- 敌人属性
 - `Enemy_health` -敌人的生命值
 - `Enemy_attack` -敌人的攻击力
 - `Enemy_speed` -敌人的速度
 - `Enemy_form` -敌人的状态
 - `Enemy_type` -敌人的攻击类型
 - `Enemy_range` -敌人的攻击范围
 - `Enemy_skill` -敌人的特殊技能
- 存储方案：Json存储+加密
- 敌人技能：
 - 此处需要单独建一张技能表
 - 此表要与敌人表建立关联，实现部分敌人的特殊技能
- 敌人AI：
 - 通过 `EntityController` 脚本，配合寻路系统算法实现敌人的自动移动
 - 寻路算法的选择后续再议
 - 根据 `Enemy_Type` 实现不同的攻击逻辑
 - 近战敌人：接近玩家并在攻击范围内攻击
 - 远程敌人：在攻击范围内向玩家发射弹幕（弹幕的控制要通过对象池）
 - `Enemy_skill`：可以根据技能描述，实现不同的技能效果（例如：召唤小怪、冲锋、范围攻击等）。
- 敌人生成：
 - `EnemySpawner` 脚本，负责在关卡中生成敌人
 - 根据策划案中的波次设定控制敌人的生成数量、类型、时间间隔
 - 可以用协程 `Coroutome` 控制波次之间的间隔
- 碰撞检测：
 - 使用Unity的碰撞系统
 - 通过玩家与敌人之间的碰撞盒碰撞来判断是否受到伤害

3. 卡牌（符咒）系统

- 策划案内容：
 - 卡组8张，初始卡片5张，等待序列3张。
 - 使用卡牌后，卡牌进入等待序列末端。
 - `Card_line=1`的卡牌有额外效果，`Card_line=5`的卡牌法力消耗+1
 - 使用卡牌需要鼠标绘制图样。
- 卡牌属性
 - `Card_name` -卡牌的名称
 - `Card_order` -卡牌的序号

- Card_line -卡牌的序列位置
- Card_cost -卡牌的法力值消耗
- Card_des -卡牌的描述
- Card_pic -卡牌的图样
- Card_type -卡牌的类别
- Card_form -卡牌的状态
- 存储方式: Json+加密
- 卡牌管理
 - 通过 CardManager 负责管理玩家的卡组、手牌和等待序列。
 - 使用 List<CardData> 来存储卡牌数据
 - 实现洗牌、抽牌、使用卡牌、移动卡牌等功能。
- UI显示:
 - 使用Unity的UI系统 (UGUI) 来显示手牌和等待序列。
 - 使用 Horizontal Layout Group 来自动排列手牌。
 - 当卡牌被使用或移动时, 更新UI显示。
- 手势识别:
 - 技术待定
- 卡牌使用:
- 当手势识别成功且法力值足够时, 触发卡牌效果。
- 根据Card_type, 实现不同的卡牌效果:
 - 伤害类卡牌: 对敌人造成伤害。
 - 效果类卡牌: 例如: 治疗玩家、增加攻击力、改变敌人状态等。

4. 宝物、道具、宠物系统

- 策划案内容: 类似于卡牌, 有各自的属性和描述。
- 存储方案同上